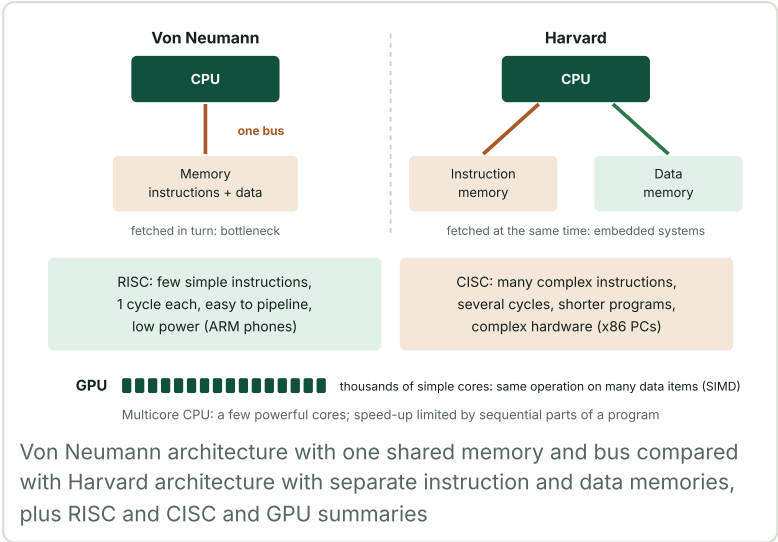
Alternative architectures: parallel, multicore, Harvard, RISC and CISC, GPUs

FD11 FACT FILE

Specification: students should be able to

- Describe parallel processing and multi-core processors, and explain their benefits and limitations
- Compare the Harvard and Von Neumann architectures and identify where each is used
- Compare reduced instruction set computers (RISC) and complex instruction set computers (CISC)
- Describe the use of graphics processing units (GPUs) and other co-processors for graphics and general-purpose parallel processing
- Evaluate the choice of processor architecture for a given application

Key words: parallel processing, multi-core processor, thread, speed-up, Von Neumann bottleneck, Harvard architecture, embedded system, microcontroller, instruction set, reduced instruction set computer (RISC), complex instruction set computer (CISC), pipelining, graphics processing unit (GPU), SIMD, co-processor



Von Neumann architecture with one shared memory and bus compared with Harvard architecture with separate instruction and data memories, plus RISC and CISC and GPU summaries

Beyond the basic design

The simple Von Neumann machine fetches and executes one instruction at a time, from one memory, over one shared set of buses. For decades performance grew mainly by raising the clock speed, but faster clocks use more power and produce more heat, and beyond a certain point a chip cannot be cooled cheaply or run from a battery for long. Designers therefore look for other ways to get more work done every second, and the alternative architectures in this lesson each tackle a different limit. Parallel processing, multi-core processors and graphics processing units carry out many operations at the same time. The Harvard architecture removes the bottleneck of a shared memory bus by giving instructions and data separate paths. Reduced instruction set computers simplify the instructions themselves so that each one finishes quickly and the pipeline keeps flowing, using less power. Real devices mix these ideas: a smartphone chip typically combines several RISC cores, a GPU and other co-processors, with separate instruction and data caches inside each core. Understanding the strengths and limits of each approach lets you explain why a games console, an office PC, a smartwatch and a washing machine contain such different processors, and choose a suitable design for a given job.

Parallel and multi-core processing

Parallel processing means using two or more processors or cores to work on different parts of a task at the same time. A multi-core processor places several complete cores on one chip, each with its own control unit, ALU, registers and level 1 cache, usually sharing a larger level 3 cache. The operating system schedules programs, or threads within a program, onto the cores so they run simultaneously, which improves multitasking and speeds up software written to divide its work. Gains depend on how much of a task can be split. Any part that must run in order, because each step needs the result of the one before, still runs on a single core, so it sets a floor on the total time. If a job takes 100 seconds and 20 of those seconds are sequential, no number of cores can bring it below 20 seconds. On top of this, dividing the work, keeping the cores coordinated and combining results all take time, and cores compete for shared cache, memory and bus. That is why a quad-core processor rarely runs a program four times faster. Parallel systems also need operating systems and programs that are written to use threads, which makes them harder to develop and test.

Harvard and Von Neumann

In the Von Neumann architecture, instructions and data share one memory and one set of buses, so they must be fetched one after the other, and the processor can be left waiting: the Von Neumann bottleneck. Its advantages are simplicity, lower cost and flexibility, since any part of memory can hold either a program or data, which suits general-purpose computers that run many different programs. The Harvard architecture uses physically separate memories and buses for instructions and for data. The processor can therefore fetch the next instruction while reading or writing data, improving speed and making timing predictable. The two memories can also be built differently: the instruction memory can be non-volatile flash of one word length, and the data memory a small amount of RAM of another. The drawbacks are extra complexity and less flexibility, because spare instruction memory cannot be used to hold data. Harvard designs are common in embedded systems, the dedicated computers inside devices such as heating controllers, car braking systems and card readers, and in digital signal processors that handle audio and video streams. Modern PCs use a hybrid: main memory follows Von Neumann, but each core has separate level 1 caches for instructions and for data.

RISC and CISC

Every processor has an instruction set, the complete set of machine-code operations it can carry out. A complex instruction set computer provides a large number of instructions, some of which perform several low-level steps in one go, such as reading two values from memory, multiplying them and storing the answer. Instructions vary in length and may take several clock cycles. CISC designs date from a time when memory was expensive and compilers were simple, so packing more work into each instruction kept programs small. A reduced instruction set computer provides a smaller set of simple, fixed-length instructions, most completing in a single clock cycle, with only load and store instructions accessing memory and everything else working on registers. A RISC program needs more instructions for the same task, and the compiler does more of the work, but the hardware is simpler. Simpler circuits use fewer transistors, consume less power and produce less heat, and uniform instructions keep a pipeline full. This is why ARM processors, which are RISC, dominate phones and tablets and now appear in many laptops, while x86 processors from Intel and AMD, which are CISC, remain common in desktop PCs largely because of compatibility with existing software. Internally, modern x86 chips break complex instructions into simpler RISC-like steps.

GPUs and co-processors

A co-processor is an additional processor that takes over a particular type of work from the main CPU. The most important is the graphics processing unit. A CPU has a small number of powerful cores with large caches and complex control logic, designed to handle varied, sequential tasks full of decisions. A GPU instead has hundreds or thousands of simpler cores that apply the same instruction to many items of data at once, an approach called SIMD, single instruction, multiple data. Graphics fits this perfectly: to draw a frame, the same lighting or colour calculation must be carried out for millions of pixels, and each pixel can be worked out independently. The same strength makes GPUs valuable outside graphics. Training machine-learning models involves enormous numbers of identical multiplications on large arrays of numbers, and video encoding, cryptography, weather forecasting and scientific simulations have similar patterns. GPU performance is often quoted in FLOPS, floating-point operations per second. GPUs are poor at sequential, branching work, so they cannot replace the CPU, which still runs the operating system and coordinates everything. Phone chips also contain other co-processors, such as image signal processors and neural engines, each optimised for one kind of task while using little power.

Choosing an architecture

No single design is best for everything; the right choice depends on what the device must do, how much power it can use, how much it can cost and what software it must run. A fitness tracker or smartwatch needs to run for days on a tiny battery, so a low-power RISC microcontroller, often using a Harvard-style memory arrangement, is ideal. A washing machine or heating controller runs one fixed program that must respond reliably to sensors, which suits an inexpensive Harvard microcontroller with its program in flash memory. An office PC must run a wide range of existing Windows applications, so a multi-core x86 processor with Von Neumann main memory remains the usual choice, although ARM-based laptops now offer longer battery life. A smartphone combines multi-core RISC processing with a built-in GPU and other co-processors to give smooth graphics and long battery life without a fan. Training an artificial intelligence model or running a weather simulation needs vast numbers of identical calculations, so GPUs or large many-core parallel systems are used. When evaluating, weigh speed for the actual workload, power and heat, cost, size and software compatibility, and remember that adding parallel hardware only helps when the task can be split.

T Key terms from the mark schemes

Parallel processing
The simultaneous use of two or more processors or cores to work on different parts of the same task.

Thread
A sequence of instructions within a program that can be scheduled to run independently, for example on a separate core.

Embedded system
A computer system built into a larger device to perform one dedicated function, such as a heating controller.
`Credited: Microcontroller (for the processor used)`

CISC
Complex instruction set computer: a processor with a large set of instructions, some of which carry out several low-level operations and take several cycles.

SIMD
Single instruction, multiple data: one instruction is applied to many data items at the same time.

Pipelining
Overlapping the fetch, decode and execute stages of successive instructions so that several are being processed at once.

Multi-core processor
A single chip containing two or more complete processing cores that can each execute instructions at the same time.

Harvard architecture
A computer design with separate memories and buses for instructions and data, so both can be accessed at the same time.

RISC
Reduced instruction set computer: a processor with a small set of simple, fixed-length instructions, most executed in one clock cycle.

Graphics processing unit (GPU)
A processor with many simple cores designed to perform the same operation on many data items in parallel, used for graphics and other parallel tasks.

Co-processor
An additional processor that takes over a specific type of work from the main CPU, such as graphics or sound.

C C# example

`Splitting work across threads in C#`

```
using System.Threading.Tasks;

int[] brightness = new int[8];

// Each image is processed on whichever core is free
Parallel.For(0, brightness.Length, i =>
{
    brightness[i] = (i + 1) * 10;   // stand-in for slow image processing
});

Console.WriteLine($"Cores available: {Environment.ProcessorCount}");
Console.WriteLine(string.Join(", ", brightness));
```

Exam focus

Comparison questions reward paired points that state both sides, for example 'RISC instructions usually take one clock cycle, whereas some CISC instructions take several'. A list of facts about only one side earns little. Link each feature to a consequence: fewer transistors lead to lower power use and longer battery life; separate buses let an instruction and data be fetched at the same time. For 'describe parallel processing', mention two or more processors or cores, working simultaneously, on different parts of the same task. For limitations of more cores, give a reason such as sequential parts of a program, software not written for threads, or competition for shared memory, rather than simply saying 'it does not always help'. When explaining why a GPU is used, mention many simple cores and the same operation applied to many data items in parallel, and link this to the task in the question. Harvard questions often ask for a typical use: embedded systems, microcontrollers or digital signal processors earn credit, but 'a desktop PC' does not. Avoid absolute claims such as 'RISC is always faster' or 'more cores always doubles speed'. In extended questions, apply each point to the scenario and finish with a justified recommendation.

? Check yourself

1. Describe what is meant by parallel processing. [2]
2. Explain two limitations of adding more cores to a processor. [4]
3. A data-analysis job takes 60 seconds on one core. 45 seconds of the work can be run in parallel and the rest cannot. Calculate how long the job takes on 3 cores, and state the shortest time it could ever take however many cores are added. [3]

Answers: 1. Two or more processors/cores are used at the same time (1) / to work on different parts of the same task/program, which is split into parts (1) 2. Any two explained, e.g.: some parts of a program are sequential (1) so they cannot be split and still run on one core, limiting speed-up (1) / software must be written to use multiple threads (1) otherwise extra cores are unused (1) / cores share cache, memory and bus (1) so they may wait for each other (1) / overhead of splitting work and combining results (1) reduces the gain (1) / more cores increase cost and power use (1) 3. Sequential part = 60 − 45 = 15 s (1) / on 3 cores: 15 + 45 ÷ 3 = 15 + 15 = 30 s (1) / shortest possible time = 15 s (the sequential part) (1)